Agent的核心架构剖析

- 来源: https://class.imooc.com/lesson/5290#mid=105149
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: Agent的核心架构剖析
- 课节: 图文 1-1 Agent的核心架构剖析

1-1 Agent的核心架构剖析

Agent的核心架构剖析

> [!TIP] 阅读指南 理解 Agent 的架构，就像理解人类的大脑结构——只有知道各个模块如何协作，才能真正掌握 Agent 的设计精髓。

经典架构：Perceive-Think-Act（PTA）

如果要用一句话概括 Agent 的核心架构，那就是：感知（Perceive）→ 思考（Think）→ 行动（Act）→ 观察（Observe）→ 循环往复。

这个模式并不是 LLM 时代的发明，而是来自经典的「智能体理论」（Intelligent Agent Theory）。早在 1995 年，Stuart Russell 和 Peter Norvig 在《人工智能：一种现代方法》中就提出了这个框架。

为什么是循环，而不是单向流程?

传统的软件是这样工作的：

```
输入 → 处理 → 输出 → 结束

1
```

比如一个计算器程序：

- 输入：2 + 3

```
2 + 3
```

- 处理：执行加法运算
- 输出：5
- 结束

但 Agent 不同。它运行在一个 动态变化的环境 中，需要不断地：

动态变化的环境

- 感知：环境发生了什么变化？
- 思考：我应该采取什么行动？
- 行动：执行决策
- 观察：我的行动产生了什么效果？
- 回到第 1 步，继续循环

Agent 循环的四个阶段

用一个具体例子来理解这个循环。假设让 Agent 准备一次周末出游：

第 1 轮：

```
[感知] 用户目标：准备周末出游
       环境信息：当前是周三，目的地未定

[思考] 分析：需要先确定目的地和天气
       计划：查询几个热门城市的天气预报

[行动] 调用工具：get_weather(cities=["杭州", "苏州", "南京"])

[观察] 结果：杭州晴天，苏州多云，南京有雨
```

第 2 轮：

```
[感知] 新信息：杭州天气最好
       用户偏好：喜欢自然景观（从历史对话记忆中提取）

[思考] 决策：推荐杭州西湖
       计划：查询景点开放时间和门票信息

[行动] 调用工具：search_attractions("杭州西湖")

[观察] 结果：西湖免费开放，周末人流量大
```

第 3 轮：

我们会发现，这个循环的关键在于：

- 每一轮都基于上一轮的结果：Agent 不是一次性输出答案，而是逐步推进；

每一轮都基于上一轮的结果

- 可以根据中间结果调整策略：如果发现南京下雨，立刻调整为推荐杭州；

可以根据中间结果调整策略

- 持续与环境交互：通过工具获取实时信息，而不是仅依赖训练数据。

持续与环境交互

PTA 的工程化实现：ReAct

上面的例子展示了 Perceive-Think-Act 循环在具体场景中的运作方式。这个经典的理论框架，到了 LLM 时代被工程化为一种更具体的实现模式——ReAct（Reasoning and Acting）。

我们在第 2 节已经提到过，ReAct 是 Agent 演进路径中第二阶段的关键模式。

从演进路径来看，Stage 2（ReAct）是当前工程实践的基线，Stage 3（自主 Agent）则是尚在探索中的目标方向。第 2 节已经说明，自主 Agent 的各项能力——自主规划、长期记忆、主动发起、自我修正——目前仍处于早期阶段，在可靠性、成本和安全性上还有大量工程问题待解决。因此，生产环境中实际落地的 Agent 系统，绝大多数仍然运行在 ReAct 模式或其精细化变体之上。理解 ReAct 架构，正是掌握当前可用 Agent 工程实践的基础。

ReAct 并没有创造一个新的循环结构，而是 把 PTA 框架中的"思考"过程从黑盒变成了白盒。

```
传统方式（思考是黑盒）：
Perceive → [Think：内部推理] → Act → Observe
                ↑
          你看不到这部分

ReAct方式（思考显式输出）：
Thought： "我需要先查询杭州的天气..."
          ↓（这段文本会被记录和展示）
Action： get_weather("杭州")
Observation： {"weather": "晴天"}
Thought： "天气不错，现在..."
          ↓
...
```

所以，PTA 和 ReAct 的关系可以这样理解：PTA 是理论框架，ReAct 是工程实现。PTA 描述了 Agent 应有的四个步骤——感知、思考、行动、观察；ReAct 则给出了在 LLM 系统中每一步具体怎么做。核心流程依然是：

```
感知输入 → 思考（现在可见） → 执行行动 → 观察结果 → 循环


1
```

ReAct 的价值：

ReAct 的价值

- 可解释：开发者能看到 Agent 的决策链

可解释

- 可调试：出错时可定位具体哪步思考有问题

可调试

- 可优化：通过分析 Thought 日志，改进 Prompt 或工具设计

可优化

它是当前最成熟、最可靠的 Agent 架构模式，也是整个第 3 节讨论各模块架构时依赖的核心框架。

从循环到模块:两套概念的关系

现在理解了 Agent 的循环流程：Perceive（感知）→ Thought（思考）→ Action（行动）→ Observation（观察）→ 循环（ReAct 模式的完整形态）。

但如果去看实际的 Agent 系统代码，会发现它们通常被组织成这样的模块架构:

- 感知层(Perception Layer)

感知层

- 规划层(Planning Layer)

规划层

- 推理层(Reasoning Layer)

推理层

- 行动层(Action Layer)

行动层

- 记忆系统(Memory System)

记忆系统

这就产生了一个问题:四步循环和五个模块,它们之间是什么关系?

四步循环

五个模块

核心区分:流程 vs 架构

两套概念的本质区别
═══════════════════════════════════

- ReAct 循环
  - 这是「流程视角」
  - 描述 Agent **做什么**
  - 关注时间顺序:先感知,再思考,再行动

- 五个模块(感知层/规划层/推理层...)
  - 这是「架构视角」
  - 描述 Agent **怎么实现**
  - 关注功能划分:每个模块负责什么

对应关系:循环步骤如何映射到模块

看看 ReAct 循环的每个步骤是由哪些模块实现的:

1. Thought(思考)被拆成了两个模块:

1. Thought(思考)被拆成了两个模块

- 规划层(Planning):制定整体计划 “我要完成写周报这个任务，需要先查数据，再提取要点，最后生成正文”
- 推理层(Reasoning):每一步的具体决策 “现在查到的数据不够，我应该调用哪个工具?用什么参数?”

为什么要拆分?

因为"思考"这个动作太复杂了:

- 有时需要宏观规划(把大任务拆成小任务)

宏观规划

- 有时需要微观决策(当前这一步怎么做)

微观决策

人类也是这样:计划"准备一次旅行"时，会先规划整体流程(订票、订酒店、查景点),然后在每个环节中推理具体操作(订哪个航班、选哪家酒店)。

2. Observation(观察)没有独立的层:

2. Observation(观察)没有独立的层

观察其实是三个动作的组合:

- 记录结果:由记忆系统保存行动的输出

记录结果

- 分析结果:由推理层评估执行效果、识别异常、提取关键信息

分析结果

假设 Agent 调用天气 API 后收到错误:

```
{"error": "城市不存在"}

1
```

但如果有"分析"环节，推理层会:

- 识别这是错误而非正常数据
- 判断需要调整策略(重新询问用户或换个城市)
- 决定下一步行动

这就是为什么观察不只是被动接收，还包含主动分析和判断。

3. 记忆系统是贯穿整个循环的:

3. 记忆系统是贯穿整个循环的

记忆系统不是循环中的某一步，而是全程参与:

全程参与

- 感知时:提取历史记忆
- 思考时:参考过往经验
- 行动后:记录执行结果

各模块详解

感知层(Perception)

职责:将外部世界的各种输入转化为LLM可以理解的文本形式。

职责

可以把感知层想象成Agent的"翻译官":它负责把来自物理世界、数字系统、用户交互的各种信息，统一翻译成LLM能理解的语言。

感知层处理的输入来源:

感知层处理的输入来源

- 用户交互:自然语言指令、图片、语音等

- 环境状态:当前时间、用户位置、系统状态等上下文信息

环境状态

- 历史记忆:从记忆系统中检索到的相关对话或经验

历史记忆

关键工作：

关键工作

1. 多模态转文本：

1. 多模态转文本

- 图片 → 文本描述（通过GPT-4V等视觉模型）
- 语音 → 文字转写（通过Whisper等语音识别）

2. 结构化数据处理：

2. 结构化数据处理

- 工具返回的 JSON 数据不一定要转成自然语言
- 关键是：让 LLM 能理解它的含义

让 LLM 能理解它的含义

例如天气 API 返回：

```
{"city": "杭州", "temp": 25, "weather": "晴"}

1
```

两种处理方式都可以：

- 直接使用 JSON：Observation: {"city": "杭州", "temp": 25, "weather": "晴"}
- 转为自然语言：Observation: 杭州当前温度25度，天气晴朗

```
Observation: 杭州当前温度25度，天气晴朗
```

转为自然语言

3. 上下文组装：

3. 上下文组装

- 将用户输入、工具返回、历史对话、可用工具列表等
- 组装成一个完整的Prompt,送给LLM处理
- 这是感知层最核心的价值:让LLM在每一步都能"看到"完整的世界

示例：

示例

用户输入： “帮我查一下明天杭州的天气”

感知层组装的 Prompt：

```
## 系统信息
当前时间：2024年12月9日 14:30
用户位置：上海

## 用户信息
用户偏好：喜欢详细的天气信息（温度、湿度、风力）

## 可用工具
1. get_weather(city, date): 获取指定城市和日期的天气
2. search_web(query): 搜索网络信息

## 用户请求
帮我查一下明天杭州的天气
```

这样，LLM 就能全面理解当前的 “环境” 了。

再看一个工具返回的例子：

再看一个工具返回的例子

假设调用 get_weather("杭州", "2024-12-10") 后得到：

```
{
  "city": "杭州",
  "date": "2024-12-10",
  "temperature": {"max": 18, "min": 8},
  "humidity": 65,
  "wind": {"speed": 12, "direction": "东北"},
  "weather": "多云"
}
```

感知层有两种选择：

方式1：保持 JSON

```
Observation: {"city": "杭州", "temperature": {"max": 18, "min": 8}, ...}


1
```

→ 优点：保持结构，如果需要传给下一个工具很方便 → 缺点：嵌套层级多时，LLM 可能难以快速提取关键信息

方式2：转为自然语言

```
Observation: 杭州2024年12月10日天气多云，温度8-18℃，湿度65%，东北风12m/s


1
```

→ 优点：LLM 更容易理解和推理，特别是需要总结时 → 缺点：如果下一步需要结构化数据，还得让 LLM 重新解析

实践中的选择原则：

实践中的选择原则

- 如果是最后一步（直接回答用户），用自然语言
- 如果还需要继续调用工具，保持 JSON
- 如果数据结构复杂（如3层以上），提取关键信息后用自然语言

职责：将高层目标拆解为可执行的子任务序列。

在最新的 Agent 架构中，规划能力 被认为是 Agent 与普通聊天机器人的最大区别。

两种规划策略：

两种规划策略

1. 一次性规划（Plan-then-Execute）

先生成完整的任务计划，再逐步执行，适合目标明确、环境稳定的场景。比如用户说"帮我写一份周报"，Agent 会先制定完整计划：

```
检索本周工作记录 → 提取关键成果和数据 → 按照模板生成周报正文 → 校对和格式化

1
```

然后按照这个计划一步步执行。

2. 边规划边执行（Interleaved Planning）

每执行一步后，根据结果重新规划下一步，这就是 ReAct 模式 的核心思想，适合环境动态变化、目标不明确的场景。

比如用户说"帮我研究一下最新的 Agent 框架"：

- 第 1 步：搜索"2025 Agent framework"，发现 CrewAI 和 AutoGen 很热门
- 第 2 步：深入查询 CrewAI 的特点，发现它强调团队协作
- 第 3 步：对比 CrewAI 和 AutoGen...

每一步都根据上一步的结果来决定下一步做什么。

职责：在每一步决策中，分析当前状态，选择最优行动。

这是 LLM 的核心能力所在。现代 Agent 的推理层通常包括两部分：

1. 知识推理：

1. 知识推理

- 基于训练数据的知识：LLM 自身的参数中存储的知识；

基于训练数据的知识

- 外部知识源：通过 RAG（检索增强生成）从向量数据库、知识图谱等获取。

外部知识源

2. 逻辑推理：

2. 逻辑推理

- Chain-of-Thought（思维链）：逐步推理，显式化中间步骤；

Chain-of-Thought（思维链）

- Tree-of-Thought（思维树）：探索多条推理路径，选择最优解；

Tree-of-Thought（思维树）

- Self-Reflection（自我反思）：检查自己的推理是否合理，必要时回溯修正。

示例：Chain-of-Thought 推理

用户问：“我应该选杭州还是南京旅游？”

Agent 的推理过程：

```
[Step 1] 首先，我需要了解这两个城市的天气情况
        → 调用 get_weather("杭州"), get_weather("南京")

[Step 2] 杭州晴天，南京有雨 → 天气角度杭州更好

[Step 3] 接下来，检索用户偏好 → 用户喜欢自然风景

[Step 4] 杭州有西湖，符合用户偏好 → 综合判断：推荐杭州
```

这种逐步推理的方式，让 Agent 的决策过程 可解释、可追踪、可调试。

可解释、可追踪、可调试

新趋势：

新趋势

- 神经符号结合（Neuro-Symbolic）： 将 LLM 的语义理解与符号推理引擎结合； 在需要精确逻辑的场景（如数学、法律）提升可靠性。
- 多轮验证机制： Agent 在给出答案后，自我验证推理是否合理； 类似于人类的“检查一遍”过程。

行动层(Action)

职责：将决策转化为具体的执行动作。

LLM Agent 的行动层与传统机器人不同，它不是控制电机转动，而是 调用工具（Tool Calling）。

常见的行动类型：

常见的行动类型

- 查询外部服务： 调用天气 API、搜索引擎、数据库等； 这是最基础、最常用的行动。
- 操作文件系统： 读写文件、创建目录、执行脚本； 需要非常严格的 权限控制。

操作文件系统：

操作文件系统

- 读写文件、创建目录、执行脚本；
- 需要非常严格的 权限控制。

权限控制

- 与其他 Agent 交互： 在 Multi-Agent 系统中，发送消息给其他 Agent； CrewAI、AutoGen 等框架的核心功能。

记忆系统(Memory)

职责：存储和检索历史信息，使 Agent 能够“记住”过去的交互。

这是目前Agent 研究的 热点领域。没有记忆的 Agent 就像“金鱼”，每次对话都是从零开始；有了记忆，Agent 才能真正成为“助理”。

两种记忆类型：

两种记忆类型

1. 短期记忆（Short-term Memory）

- 定义：当前对话的上下文；

定义

- 存储方式：直接在 LLM 的上下文窗口中；

存储方式

- 生命周期：单次会话结束后清空。

生命周期

```
用户：我喜欢吃川菜
（存入短期记忆）

Agent：好的，我记住了。

用户：帮我推荐一家餐厅
Agent：根据您喜欢川菜，我推荐...
（从短期记忆中提取）
```

挑战：

挑战

- 上下文窗口有限（如 GPT-4 的 128K tokens）；
- 对话太长时，早期信息会被截断。

2. 长期记忆（Long-term Memory）

- 定义：跨对话的持久化信息；
- 存储方式：外部数据库（向量数据库、知识图谱、关系数据库）；
- 生命周期：持久存在，需要时检索。

核心技术：RAG + 向量数据库

目前，向量数据库 被认为是 Agent 记忆系统的最佳选择：

向量数据库

```
[存储阶段]
用户信息："我喜欢吃川菜"
→ Embedding 模型转化为向量
→ 存入向量数据库（如 Pinecone, Weaviate, Chroma）

[检索阶段]
用户问："帮我推荐餐厅"
→ 将查询转为向量
→ 在数据库中搜索相似向量
→ 检索到："用户喜欢川菜"
→ 将这条信息添加到当前 Prompt
```

新趋势：Agentic RAG

> Agentic 是什么? 现在，你会发现很多技术都加上了"Agentic"前缀:Agentic RAG、Agentic Workflow、Agentic Search... Agentic = 赋予 Agent 特性,核心含义是: 从"被动执行"变为"主动决策" 从"一次性处理"变为"多轮迭代" 从"固定流程"变为"自主规划" 简单来说，给任何技术加上Agentic,就是让它具备Agent的自主性和循环能力。

传统 RAG 是"一次性检索":Agent 提问 → 检索 → 生成答案。

Agentic RAG是"多轮自主检索":

```
第 1 轮：检索到一些信息
Agent：这些信息不够，我需要更多关于 X 的内容

第 2 轮：重新检索，调整查询
Agent：现在信息足够了，可以生成答案
```

这种模式让 Agent 对知识的获取更主动、更灵活。

下节预告

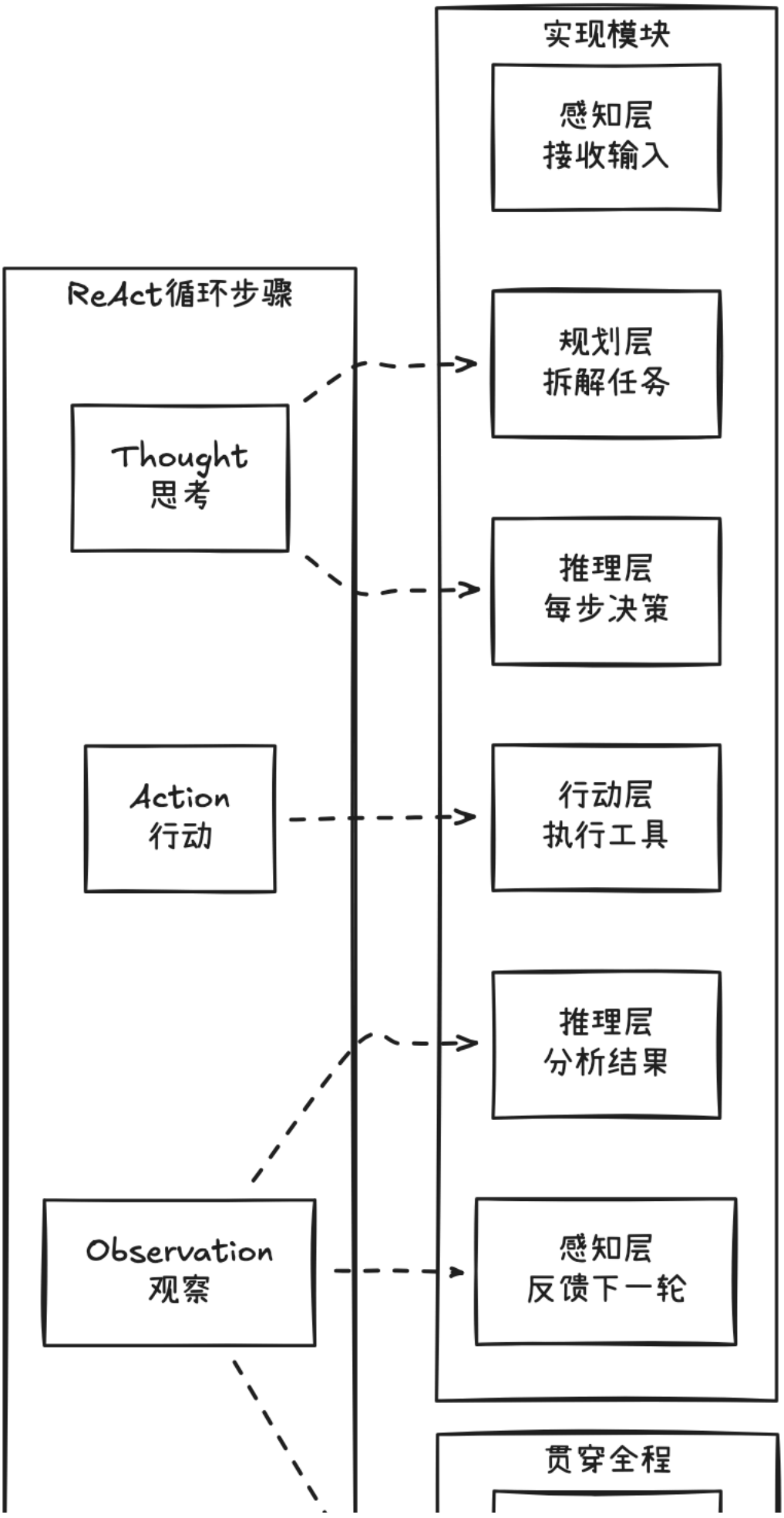
现在了解了 Agent 的模块架构，但可能会好奇：这些模块背后的核心能力到底是如何实现的？感知如何理解环境？思考如何进行推理？行动如何准确执行？

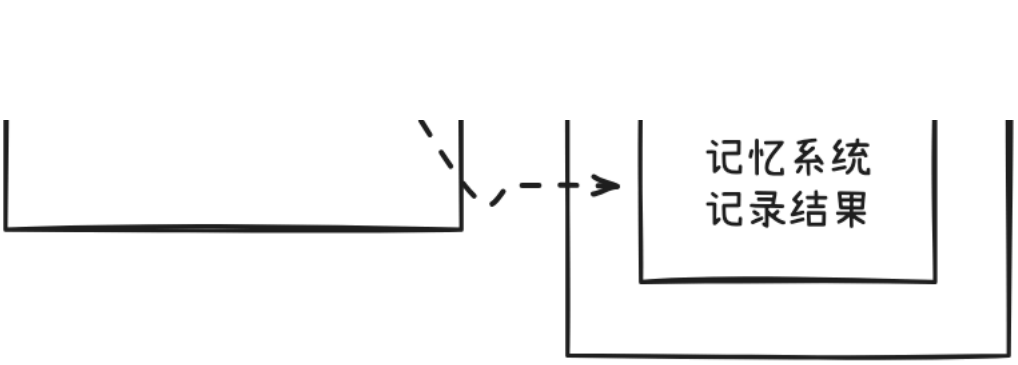
下一节，我们将深入剖析 Agent 的三大核心能力——从多模态感知到 Chain-of-Thought 推理，再到工具调用的技术细节，可以看到，原来 Agent 的"智能"是通过这些具体的能力实现的。

■ 学点英语

中文	English	音标	说明
感知-思考-行动	PTA (Perceive-Think-Act)	/piː tiː eɪ/	Agent运行的核心理论循环框架
感知层	Perception Layer	/pərˈsepʃn ˈleɪər/	将外部输入转化为LLM可理解形式的模块
规划层	Planning Layer	/ˈplænɪŋ ˈleɪər/	将高层目标拆解为可执行子任务序列的模块
推理层	Reasoning Layer	/ˈriːzənɪŋ ˈleɪər/	每一步决策中分析当前状态选择最优行动的模块
行动层	Action Layer	/ˈækʃn ˈleɪər/	将决策转化为具体执行动作的模块
自主检索增强生成	Agentic RAG	/eɪˈdʒentɪk ræg/	多轮自主检索，Agent主动调整查询策略的知识获取方式

图片


实现模块
感知层
接收输入
ReAct循环步骤
规划层
拆解任务
Thought
思考
推理层
每步决策
Action
行动
行动层
执行工具
推理层
分析结果
Observation
观察
感知层
反馈下一轮
贯穿全程


记忆系统
记录结果